

Escuela Politécnica Nacional
Facultad de Ingeniería en Sistemas

Mini-Informe
Ejercicio 4

Nombre
Chávez Estefano
Magarisca Bryan
Tixicuro Daniel

Asignatura
Compiladores y lenguajes

Docente
Ing. Rodrigo Chancusig MSc.

Carrera
Ingeniería en Ciencia de Datos e Inteligencia Artificial

Paralelo
GR1CD

Fecha
10 de noviembre del 2026

Ejercicio 4 - Analizador Léxico Básico con Lex/Flex (Identificadores y Exponenciales)

Objetivo

Desarrollar y explorar el funcionamiento de un analizador léxico mediante el uso de la herramienta Lex, con el propósito de compilar un programa con la capacidad de identificar patrones específicos: números, identificadores y expresiones exponenciales dentro de un flujo de caracteres, ejecutando acciones de notificación.

Preparación del Entorno y Estructura de Archivos

Para iniciar la práctica correspondiente al Ejercicio 4, lo primero que se realizó fue la preparación del entorno de trabajo en nuestro sistema operativo basado en Linux. Se trabajó dentro de un directorio específico designado para la materia, con el fin de mantener un orden secuencial y adecuado. En este directorio, al finalizar toda la práctica, se alojarán todos los archivos necesarios para el funcionamiento del programa, los cuales son los siguientes:

- Código fuente del analizador (cuatro.l)
- Archivos de texto de prueba (entrada.txt y salida.txt)
- Archivos generados automáticamente por los compiladores (lex.yy.c y a.out)

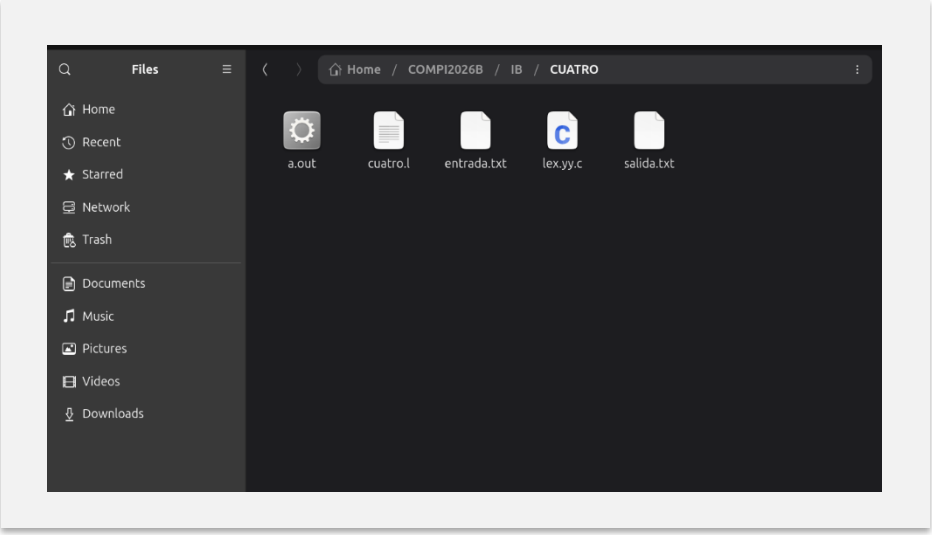


Figura 1. Contenido del directorio CUATRO.

Desarrollo del Código Fuente (cuatro.l)

Para esta práctica, el nucleo principal se basó en un analizador léxico, y para ello, se utilizó un código fuente cuya extensión es .l, el cual va a tener el nombre de “cuatro.l”. Este archivo se divide en secciones principales y fundamentales en la estructura Lex:

- **Sección de Declaraciones:** En este apartado se incluye la librería estándar de C, denominada como <stdlib.h>, necesaria para la gestión de memoria y utilidades del programa como printf.
- **Sección de Reglas:** Se define lo que debe hacer el programa cuando encuentre ciertas secuencias de caracteres. Para este caso, se definieron tres reglas principales:
    1. **[0-9]+ :** Identifica secuencias de uno o más dígitos y ejecuta printf("NUMERO \n");.
    2. **[a-zA-Z][a-zA-Z0-9]*** : Identifica cadenas que inician con una letra seguida de letras o números, ejecutando printf("IDENTIFICADOR \n")

Para este ejercicio se añadió un reconocedor de números reales en notación exponencial.

3. [+-]?[0-9]+(\.[0-9]+)?[E|e][+-]?[0-9]+ : Esta es la regla más compleja del código ya que permite identificar números en notación científica o con punto flotante. Su funcionamiento se desglosa de la siguiente manera:

- [+-]?: Detecta un signo positivo o negativo opcional al inicio.
- [0-9]+: Asegura que exista al menos un dígito entero.
- (\.[0-9]+)?: Identifica de manera opcional la parte decimal (un punto seguido de más dígitos).
- [E|e][+-]?[0-9]+: Identifica la parte exponencial, buscando la letra 'E' o 'e', seguida de un signo opcional y un dígito final para el exponente. Al cumplirse todo el patrón, se ejecuta printf("EXPONENCIAL \n")

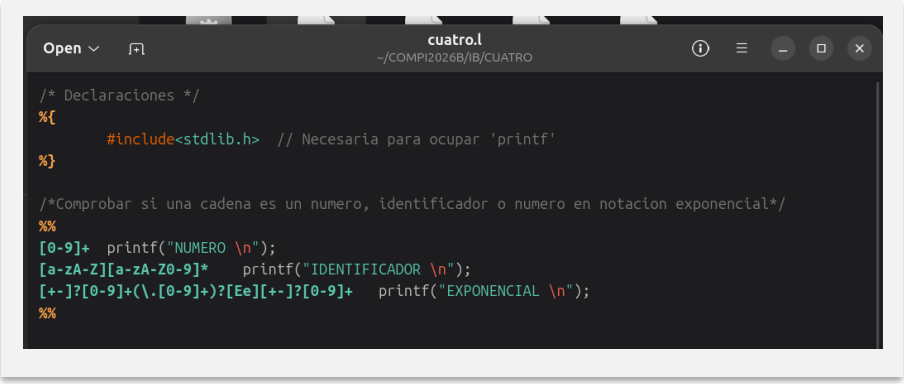


Figura 2. Código fuente de “cuatro.l”

Proceso de Compilación y Ejecución Interactiva

1. **Generación del código en C:** Una vez que se ejecuta el comando “lex cuatro.l”, Lex lee las reglas del documento de texto y genera un archivo en C denominado como ***lex.yy.c.***
2. **Compilación del ejecutable:** Se utiliza GCC mediante el comando “gcc lex.yy.c -lfl”, en donde la parte final **-lfl** es un elemento importante para enlazar la librería de Flex, resultando de esta manera en el ejecutable ***a.out.***

Una vez que se ejecuta el a.out mediante el comando ./a.out, el programa detecta las cadenas ingresadas y responde clasificándolas como números, identificadores o exponenciales.

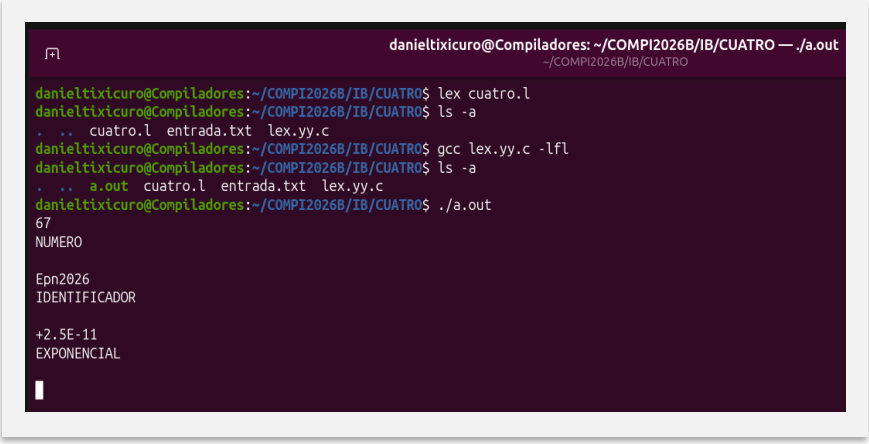


Figura 3. Proceso de compilación y ejecución del analizador de numeros, identificadores y numeros en notacion exponencial resultante.

Pruebas con Archivos y Redirección de Flujos (Entrada/Salida)

Para simular un ambiente más parecido a un entorno real, se probó el programa funcional ingresando archivos de texto completo en lugar de tipear manualmente en la terminal.

- **Archivo de entrada:** Se redactó el archivo "entrada.txt", en el cual se redacto una lista con caracteres que el programa no reconoce con el fin de usarlo como indexadores.



Figura 4. Contenido correspondiente al archivo de texto "entrada.txt"

- **Redirección de entrada y salida:** Se ejecutó el comando ./a.out < entrada.txt > salida.txt, donde el símbolo < inyecta el archivo de texto previamente creado como una entrada estándar, mientras que el símbolo > captura la salida estándar del programa y la guarda en un archivo de texto nuevo llamado "salida.txt".

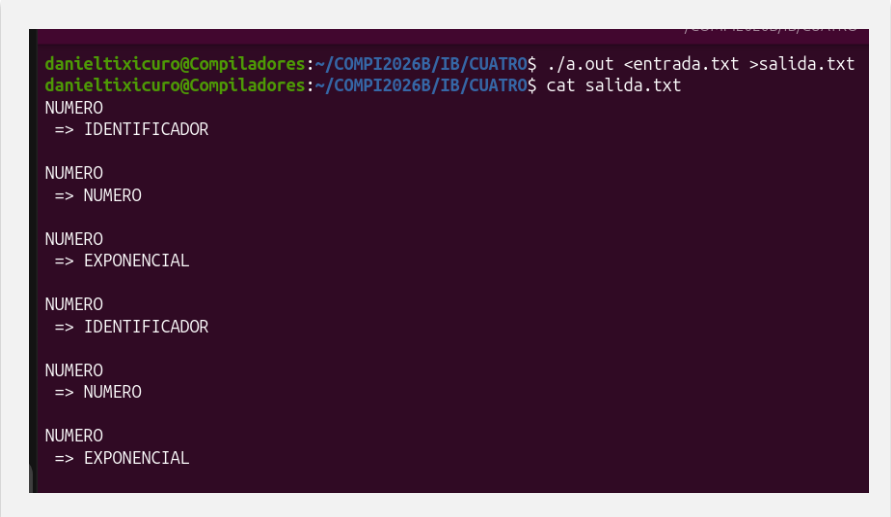


Figura 5. Ejecución de "a.out" con redireccionamiento de entrada y salida

Conclusión

En este ejercicio práctico se ha demostrado la eficiencia y simplicidad que ofrece Lex para la creación de herramientas que requieren identificar patrones especificos, pues mediante la definición de expresiones regulares básicas y avanzadas, fue posible automatizar la detección y respuesta ante patrones como la notación científica, identificadores y enteros en el flujo de datos continuo. Además, se comprobó que los programas generados por Lex pueden integrarse sin problemas con las funcionalidades nativas del sistema operativo Linux, como por ejemplo los flujos de entrada y salida (stdin/stdout), otorgando la capacidad de procesar archivos de texto de manera limpia y eficaz.